

# METHODS

**Object-Oriented Programming**



**Marvin C Santos**  
Instructor

## What is a Method?

---

- Is a block of code which only runs when it is called
- A code block that ***contains a series of statements***
- Are used to perform certain actions, and they are also known as **functions**.
- It defines the behavior of the objects created from the class
- Its an action that an object is able to perform

## Why use Method?

---

- To reuse code: define the code once, and use it many times. ***Code reusability***
- You can pass data (parameters)
- Can used to perform certain actions.
- Help to manage the complexity of the Program by dividing a bigger complex task into smaller, easily understood task.
- Useful in hiding the implementation details

## Basic Syntax of Method

---

```
[access modifier] [return type]  
MethodName ([parameters])  
{  
    Method body (statements)  
}
```

```
public int Add (int a, int b)  
{  
    return a + b;  
}
```

# Method Components

---

- **myMethod()** – Name of method
- **private, public, protected.** – Access modifiers
  - Keywords that are used in order to specify the accessibility of the methods.
- **void** – Return Type
  - Means that this method does not have a return value.





# Access Modifiers

- controls who can access the method

- **public** –

- Public members can be directly called once the object is created.

- **private** –

- Private members called within a method and it is not visible when object is created.

***“The idea is to containerized and encapsulate as much as you can”***



## Calling a Method

---

- Defining a method does **NOT** execute it.
- The method runs when another part of the program calls it.
- Use parentheses () when calling a method.

```
1 reference
static void Greet() { Console.WriteLine("Hello!"); }
0 references
static void Main(string[] args)
{
    Greet();    // method call
}
```

## Void Methods

---

- A void method performs an action but does not return a value.
- *Common examples:* displaying text, changing an object, or performing a procedure.

```
0 references
static void Main(string[] args)
{
    static void DisplayWelcome()
    {
        Console.WriteLine("Welcome to C# OOP!");
    }
}
```



## Method Overloading

- C# allows multiple methods to have the same name but with different parameter lists.

```
1 > public int Add(int a, int b) ...  
5  
6 public double Add(float a, float b)  
7 {  
8     return a + b;  
9 }  
10
```

## Static vs. Instance Methods

---

- **Static method** – belongs to the class and can be called without creating an object.
- **Instance method** – belongs to an object and normally requires an object instance.

## Static Methods

---

```
public static void SayHello()  
{  
    Console.WriteLine("Hello!");  
}
```

- No object needed, Just use the class name.
- Can be called without creating an object of the class.

```
Student.SayHello();
```

## Instance Methods

---

```
public void Study()  
{  
    Console.WriteLine("I am studying.");  
}
```

- First, create an object.
- Then use the object to call the method.

```
Student student = new Student();  
student.Study();
```

## Static Methods

---

- Can be called without creating an object of the class
- It belongs to the class rather than instances of the class

```
1  public static void Greetings()  
2  {  
3      Console.WriteLine("Welcome to PLV...!");  
4  }  
5
```

```
ClassName.Greetings();
```



## Optional Parameters

---

- C# allows specifying *default values* for parameters, which makes them optional when calling the method.

```
5 public void StudentDetails(string school = "PLV", string name)
6 {
7     Console.WriteLine($"{school} - {name}");
8 }
```

```
StudentDetails("Marvin Santos",)
```

```
// PLV - Marvin Santos
```

# **C#**

## **Built-in Methods**



---

# Math Functions

## Math Functions

---

- **Round():** Rounds a number to the nearest integer or specified number of decimal places.

```
26    double rounded = Math.Round(9.25645, 2);
```

```
// 9.25
```

## Math Functions

---

- **Sqrt():** Returns the square root of a number.

```
30 double sqrt = Math.Sqrt(16);
```

```
// 4
```

## Math Functions

---

- **Pow()**: Returns the value of x to the power of y.

```
33  double result = Math.Pow(2, 5);
```

```
// 32
```

## Math Functions

---

- **Abs():** Returns the absolute value of a number.

```
35    int absValue = Math.Abs(-18);
```

```
// 18
```

- 
- **Console.WriteLine():** Display Output

```
Console.WriteLine("Hello, Students!");
```



---

# Functions

## Common Mistakes

# Forgetting to Call the Method

- A method will not run automatically just because we created it.

```
using System;

0 references
class Program
{
    0 references
    static void SayHello()
    {
        Console.WriteLine("Hello!");
    }

    0 references
    static void Main()
    {
        SayHello();    // Call the method
    }
}
```

# Using the Wrong Number or Type of Arguments

- The number and type of arguments must match the method parameters

```
static void Greet(string name)
{
    Console.WriteLine("Hello " + name);
}
```

```
Greet();           // Missing argument
Greet("Marvin", "Sir"); // Too many arguments
Greet(100);        // Wrong type
Greet("Marvin");
```

# Forgetting return in a Non-Void Method

- If a method says it will return an int, it **must** return an integer.

```
static int Add(int a, int b)
{
    int result = a + b;
    return result;
}
```

\*\*\* *The compiler will complain because the method promised to return an int.*

```
int answer = Add(10, 20);

Console.WriteLine(answer);
```

# Trying to Access a private Method Outside Its Class

- A **private** method can only be used inside the same class

```
class Student
{
    1 reference
    private static void ShowName()
    {
        Console.WriteLine("Marvin Santos");
    }
}
```

0 references

```
class Program
```

```
{
    0 references
    static void Main()
    {
        Student.ShowName();
    }
}
```

// ERROR

---

## A GOOD method should do one clear job

```
CalculateTotal();  
DisplayResult();  
SaveData();
```

*This is easier to understand than having one giant method that does everything.*

## Key Takeaways

---

1. *Methods make programs modular, readable, reusable, and easier to maintain.*
2. *Methods can accept parameters and may return values.*
3. *void methods perform actions without returning a value.*
4. *Method overloading allows the same method name with different parameter lists.*
5. *In OOP, methods represent the behaviors or actions of objects.*

---

# String Manipulation Functions





## String Manipulation Functions

---

- **Length:** Gets the number of characters in a string.

```
4  string greet = "Hello World";  
5  int length = greet.Length;
```

## String Manipulation Functions

---

- **ToUpper()** / **ToLower()**: Converts a string to upper or lower case format.

```
7  string greet = "Hello World";  
8  string upper = greet.ToUpper();  
9  string lower = greet.ToLower();
```

```
// "HELLO WORLD"  
// "hello world"
```

## String Manipulation Functions

---

- **Contains():** Checks if a string contains a specific substring

```
12  string greet = "Hello World";  
13  bool contains = greet.Contains("lo");
```

```
// true
```

```
15  bool contains = greet.Contains("flo");  
16
```

```
// false
```

## String Manipulation Functions

---

- **Replace():** Replaces all occurrences of a substring with another string.

```
18  string greet = "Hello World";  
19  string replaced = greet.Replace("l", "z");  
20
```

```
// "Hezzo Worzd"
```

## String Manipulation Functions

---

- **Substring()**: Retrieves a portion of the string.

```
22  string greet = "Hello World";  
23  string sub = greet.Substring(0, 3);
```

```
// "Hel"
```

---

# Array Functions



## Array Functions

---

- **Length:** Returns the number of elements in the array.

```
38  int[] grades = { 82, 92, 81 };  
39  int length = grades.Length;
```

```
// 3
```

## Array Functions

---

- **Sort():** Sorts the elements of an array in ascending order.

```
42  int[] grades = { 82, 92, 81 };  
43  Array.Sort(grades);
```

```
//81,82,92
```



## Array Functions

---

- **IndexOf():** Returns the index of the first occurrence of an element. *The elements of an array in ascending order.*

```
45  int[] grades = { 82, 92, 81 };  
46  int index = Array.IndexOf(grades, 92);
```

```
// 1
```

---

# Type Conversion Functions



---

**PARSE** = works ONLY on string to specific data types

**string TO int**  
**string TO double**  
**string TO bool**

**CONVERT** = used to change a value from one data type to another

**int TO string**  
**string TO float**

## Type Conversion Functions

---

- **ToInt32()**
- **.ToDouble()**
- **ToString():** Converts an object to the specified type.

```
49    int number = Convert.ToInt32("12345");
```

## Type Conversion Functions

---

- **Parse()**: Converts a **string** to a specific type.

```
56    int parsedNumber = int.Parse("12345");
```

```
string number = "100";
```

```
int result = int.Parse(number);
```

```
Console.WriteLine(result + 50);
```

**Thank You!**